Xiyao Long A note on Distributed programming

Local computing and distributed computing in a huge part differs in that distributed computing has the latency problem. This difference dictates how should we design distributed computing application that meets the performance requirement. This usually comes down to see whether objects are local or remote and whether they needs to be clustered together eventually.

The author made me aware that a distributed computing paradigm is different from programing models. Also, for each programming model such as "object oriented" or "procedural calls" we need to tweak the distributed computing paradigm to adjust to the structure and mechanism of that specific model. And the authors thinks that a lot of progress in the field aim at a programming model that can naturally be used by distributed computing and its design details is inherently friendly to distributed computing application.

The authors, however, also pointed out that programmers found out despite the efforts in making the unification of the local and distributed paradigm and general programming models, the existing ones are not easy and sufficient. This is because the issues of two machines have different memory access paradigms and more generally the problem of concurrency. For example, different machines have different address spaces and have to be either dealt by having a unified system handling all address or by having programmer explicitly handling remote and local objects differently. People come to realize that a more "coarser grained" solution need to be provided to relieve individual programmers to handling all issues.

I personally actually disagree with the authors claim of "removing address-space-relative pointers" and using "system defined references" will shun beginning distributed-programming programmer. I think it's should be fairly easy for them to pick up the distinction. If eliminating the pointers can easily solve the memory access problem between different machines and prevent errors, then it should be the solution. The authors however did mention that masking the local and remote address access is discouraging programmers from distinguishing their application components/objects based on if they are local or remote, therefore unaware of the performance problem and "partial error" handling.

Another essential problem the author mentioned is that the handling of concurrency has to be dealt with by both the interface and by the object handling in the programming model. And the objects need to utilize identity(local or remote) specific syntax regardless of they are local or not. Expanding on this, authors suggests a paradigm which implement strict distinction between local objects and remote objects. One type will not know the implementation details of the other type and they different in the memory and failure handling mechanism.

Overall, I think the authors developed their point of view pretty well as they talk about that memory issues and indeterministic nature of errors in the system demands different interface for treating remote objects differently but consistently. They not only reviewed status quo but bring up a fairly clear suggestion.

Reference:

Waldo J, Waldo J, Waldo J, Wyant G, Wyant G, Wollrath A, Wollrath A, Kendall S, Kendall S. 1994. A Note on Distributed Computing. *IEEE MICRO* 17:49--64.